

Reading the Documentation: FPGA and MCU

Lecture 06

Josh Brake
Harvey Mudd College

Outline

- FPGA documentation overview
- FPGA documentation Bingo
- Manual design mapping exercise
- The same skill on a new chip: STM32L432KC documentation
- The Cortex-M4 architecture and memory map
- C refresher: turning an address in the manual into a register you can write

Learning Objectives

By the end of this lecture you should be able to...

- Find basic specs in the FPGA documentation to answer questions about your system like DC logic levels, max clock speed, etc.
- Determine the number of logic cells required by simple Verilog modules.
- Name the four STM32 documents and say which question each one answers.
- List the basic details of the architecture of the ARM Cortex-M4 CPU used in our STM32 MCU.
- Turn a base address and offset from the reference manual into C that reads and writes that register.

Part 1: Reading the FPGA Documentation

Lattice iCE40UP Documentation

Documentation

Quick Reference		Technical Resources		Information Resources			Downloads	
Key Documents Data Sheet Application Note Pin & Package User Manual		Reference Design Product Change Notification Schematic		Product Brochure Quality Assurance White Paper			IBIS Model	
		TITLE 		NUMBER	VERSION	DATE	FORMAT	SIZE
		iCE40 UltraPlus Family Data Sheet 		FPGA-DS-02008	2.0	9/20/2021	PDF	1.1 MB
		Memory Usage Guide for iCE40 Devices 		FPGA-TN-02002	1.7	10/14/2020	PDF	954.3 KB
		iCE40 I2C and SPI Hardened IP Usage Guide 		FPGA-TN-02010	1.7	9/11/2020	PDF	1.3 MB
		iCE40 SPRAM Usage Guide 		FPGA-TN-02022	1.3	4/16/2021	PDF	912.9 KB
		iCE40 Hardware Checklist 		FPGA-TN-02006	2.0	3/10/2022	PDF	355.4 KB
		iCE40 Oscillator Usage Guide 		FPGA-TN-02008	1.7	1/25/2021	PDF	675 KB
		iCE40 sysCLOCK PLL Design and User Guide 		FPGA-TN-02052	1.4	4/30/2022	PDF	1.3 MB
		iCE40 LED Driver User Guide 		FPGA-TN-02021	1.5	11/29/2021	PDF	2 MB

<https://www.latticesemi.com/en/Products/FPGAandCPLD/iCE40UltraPlus>

Lattice iCE40UP Documentation

- Info on FPGA chip itself
 - DC logic levels
 - Timing information
 - Package dimensions
 - Pinout information
 - Block diagrams of internal components



iCE40 UltraPlus Family Data Sheet

Data Sheet

FPGA-DS-02008-2.0



Package Diagrams

Data Sheet

FPGA-DS-02053-6.8

UPduino Documentation

Specs specific to the UPduino board

- Schematics
- Supporting hardware
- Programming instructions

UPduino Documentation

tinyVision.ai

UPduino v3.0: PCB Design Files, Designs, Documentation

The UPduino v3.0 is a small, low-cost FPGA board. The board features an on-board FPGA programmer, flash and LED with _all_ FPGA pins brought out to easy to use 0.1" header pins for fast prototyping.

The tinyVision.ai UPduino v3.0 Board Features:

- Lattice UltraPlus ICE40UP5K FPGA with 5.3K LUTs, 1Mb SPRAM, 120Kb DPRAM, 8 Multipliers
- FTDI FT232H USB to SPI Device
- _ALL_ 32 FPGA GPIO on 0.1" headers
- _ALL_ FTDI pins brought to test points
- 4MB SPI Flash
- RGB LED
- On board 3.3V and 1.2V Regulators, can supply 3.3V to your project
- Open source schematic and layout using KiCAD design tools
- Integrated into the open source [APIO toolchain](#)

FPGA Documentation Bingo

Bingo Card

#	Question	Answer	Reference
1	Recommended operating voltage for VCCIO	1.71-3.46 V	Table 4.2, p. 29
2	I/O pin input capacitance	C1 = 6 pF	Table 4.5, p. 32
3	3.3 V LVCMOS Logic Levels	VIL (-0.3, 0.8), VIH (2.0, VCCIO + 0.2), VOL 0.4, VOH VCCIO - 0.4	Table 4.13, p. 34
4	Recommended core input voltage	1.14-1.26 V	Table 4.2, p. 29
5	Maximum speed of sysI/O buffer	250 MHz	Table 4.19, p. 36
6	Maximum PLL output frequency	275 MHz	Table 4.22, p. 38
7	Maximum output current	+/- 8 mA	Table 4.13, p. 34
8	Recommended operating junction temperature range (industrial grade)	-40-100 °C. Note this is <i>not</i> the absolute max rating, which is -65-125 °C.	Table 4.2, p. 29 (Table 4.1, p. 29 for abs. max)
9	Input leakage current	III, IIH = 10 µA	Table 4.5, p. 32
10	What FPGA device is on the UPduino v3.1 board?	ICE40UP5K-SG48ITR	UPduino schematic
11	Major elements in a Logic Cell (LC)	4-input LUT, 1-bit DFF, Carry logic	Figure 3.2, p. 12
12	What is the range of clock frequencies for the high speed internal oscillator?	48 MHz, +/-20%	Table 4.11, p. 34
13	How many logic element are on our FPGA?	5280	Table 2.1, p. 9; multiple

Bingo Card

#	Question	Answer	Reference
14	How many EBR memory blocks?	30,120 Kbits total	Table 2.1, p. 9
15	How many SPRAM memory blocks?	4, 1024 Kbits total	Table 2.1, p. 9
16	What is the difference between EBR and SPRAM?	EBR can be configured at boot, SPRAM cannot.	
17	How many I/O banks?	3 (0, 1, 2)	Figure 3.9, p. 24
18	Data buss skew across a bank of I/Os?	510 ps	Table 4.21, p. 37
19	Pin-to-pin propagation delay through one 4-input LUT?	9 ns. This is tPD, which includes both I/O buffers – not the LUT on its own.	Table 4.21, p. 37
20	PIO input register setup time	-0.5 ns	Table 4.21, p. 37
21	PIO input register hold time	5.55 ns	Table 4.21, p. 37
22	PIO output register clock to output (Q)	10 ns	Table 4.21, p. 37
23	Number of multipliers in the FPGA	8 DSP blocks, each a single 16-bit x 16-bit or two 8-bit x 8-bit multipliers	Table 2.1, p. 9; Section 3.1.7, p. 18
24	Package for our FPGA chip	SG48 (48-pin QFN 0.50 mm pin pitch)	UPduino Documentation
25	How many user I/Os does the FPGA chip have?	39	Table 2.1, p. 9; multiple
26	What is the area of the FPGA package?	7 mm x 7 mm	Package Diagrams document, p. 36
27	What voltage does the FPGA core run on?	1.2 V	Table 4.2, p. 29

Manual Design Mapping

Going from Verilog HDL to logic cells

- 2-input AND: 1 LE
- 4-input AND: 1 LE
- 5-input AND: 2 LE
- 16-input AND: 5 LEs
- Arbitrary function of 5 inputs: 3 LEs ($f(0)$, $f(1)$, select between)
- Arbitrary function of 6 inputs: 7 LEs (4 functions of 4 inputs, two levels of 2:1 mux to choose output from the four individual 4-input LUT.)
- 2 inverters: 2 LEs
- Divide by 3 counter: 2 LEs (2 flops + 2 functions of 2 inputs)

Part 1 Wrap Up

- FPGA documentation contains important information like logic levels and timing specs
- Learning to navigate the documentation is a skill that must be practiced. Try to browse instead of search.
- As a hardware designer, you should be able to explain how many logic cells your design costs, and why, before the tools tell you.

Part 2: The STM32L432KC

The Same Skill, A New Chip

Nothing you just did was really about Lattice. Every vendor splits the same information across the same kinds of documents.

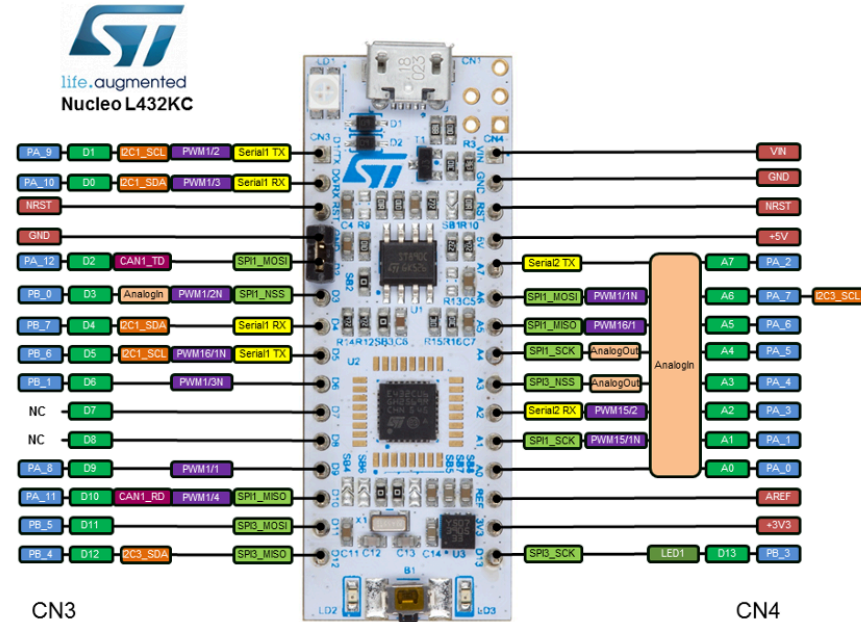
Question you have	On the FPGA	On the MCU
What are the logic levels and timing specs?	iCE40 UP Family Data Sheet	DS11451 Datasheet
What is inside the chip and how do I configure it?	Technology Library	RM0394 Reference Manual
What does the CPU itself do?	(no CPU)	PM0214 Programming Manual
How is the dev board wired?	UPduino documentation	UM1956 Nucleo-32 manual

The reference manual is 1600 pages. You are never going to read it. You are going to *navigate* it, exactly like you just navigated the datasheet.

STM32 Nucleo-32 board

Components

- MCU – STM32L432KC
- External flash memory
- 24 MHz crystal oscillator
- On-board ST-LINK debugger/programmer. Virtual COM port and debug port.
- 1 user LED and 1 reset push button
- Arduino Nano V3 form factor



What is an MCU

- MCU = MicroController Unit
- Processor core + peripherals

Figure 1. System architecture

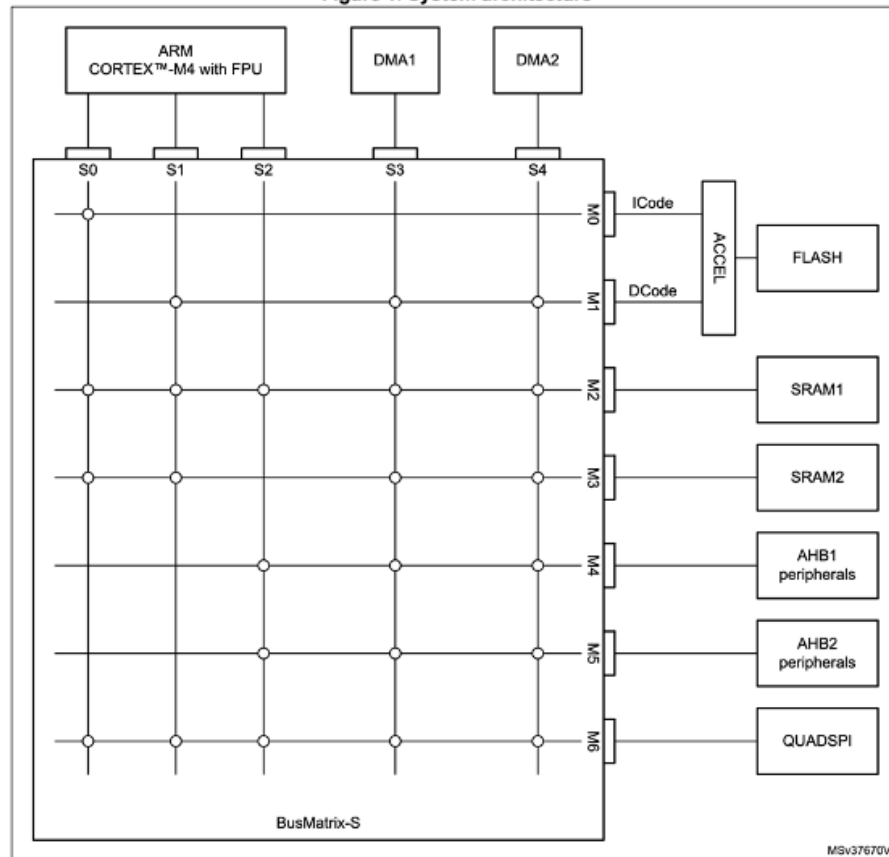
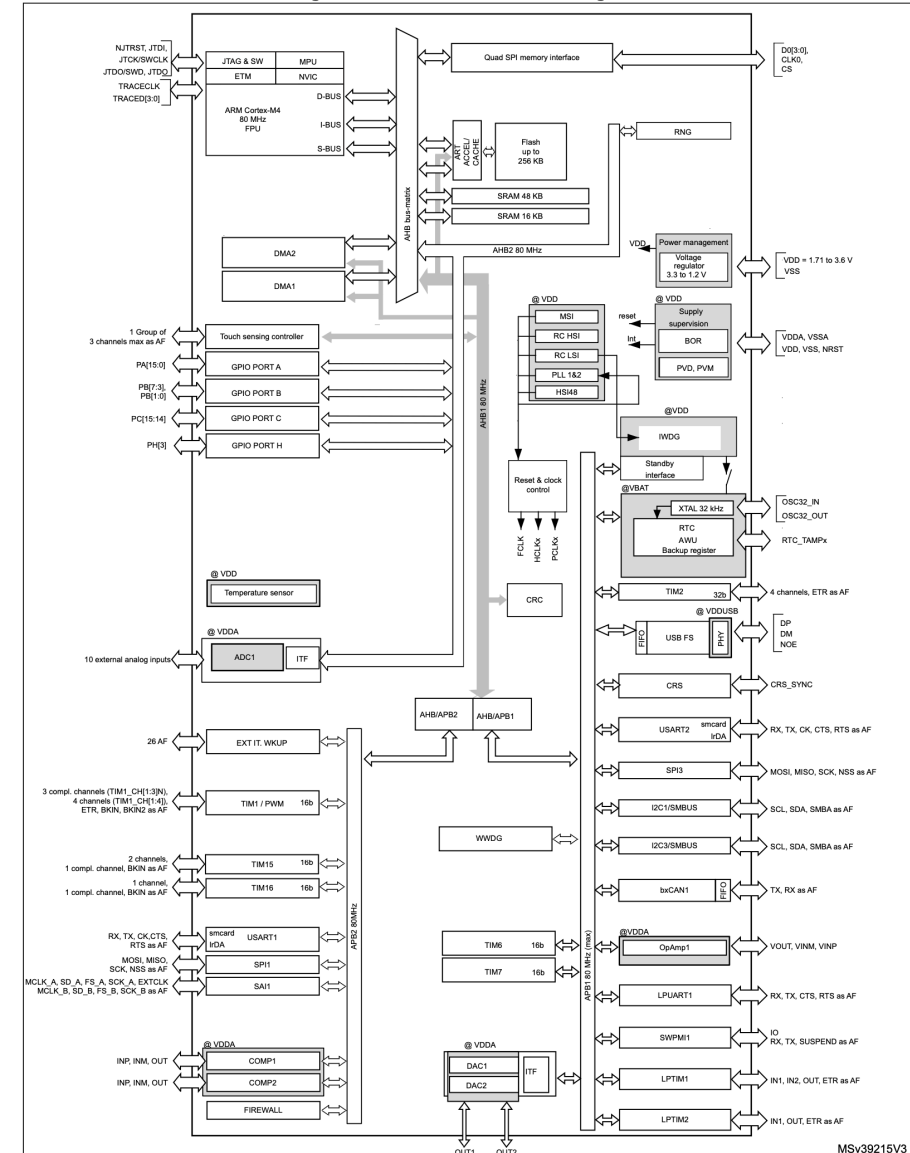


Figure 1. STM32L432xx block diagram



Note: AF: alternate function on I/O pins.

Documentation

Reference Manual: Information about all peripherals and their control registers

Datasheet: System block diagram, Pin functions, electrical characteristics, timing specs

Programmer's Manual: Information about the architecture (e.g., Cortex-M4), supported assembly instructions, registers, memory map, etc.

Questions for a new MCU

- What is the **register** set?
- What does the **memory map** look like?
- What **addressing modes** are used?
- What types of **instructions** exist?
- What **I/O functions** are available?

STM32 L432KC Architecture

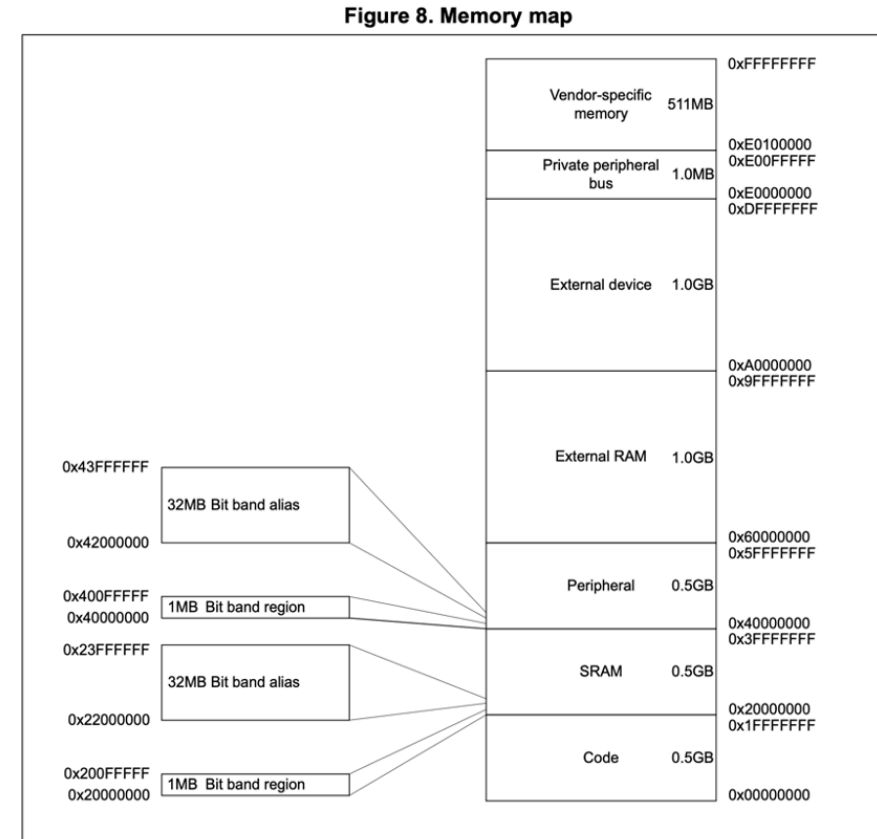
- STM32 MCU has an ARM Cortex-M4.
- It runs the ARMv7E-M architecture. This is a 32-bit architecture.
- Also supports Thumb-2 execution. Thumb-2 is a set of compressed, 16-bit instructions.
- One special thing to watch with Thumb is conditional execution. With Thumb execution you can only use conditional execution within an “if-then” block which can hold up to four successive instructions.

Architecture Overview

- Instructions are 32-bit
- Sixteen 32-bit registers R0-R15
- Supports condition codes
- Most instructions operate on two registers and put result in a third.

Memory Map

- Flat 32-bit instruction set
- Addressed in bytes.
- 2^{32} bytes of memory accessible (4 Gigabytes)
- Instructions are always aligned on word (4-byte) in standard ARM and halfword (2-byte) boundaries in Thumb mode.

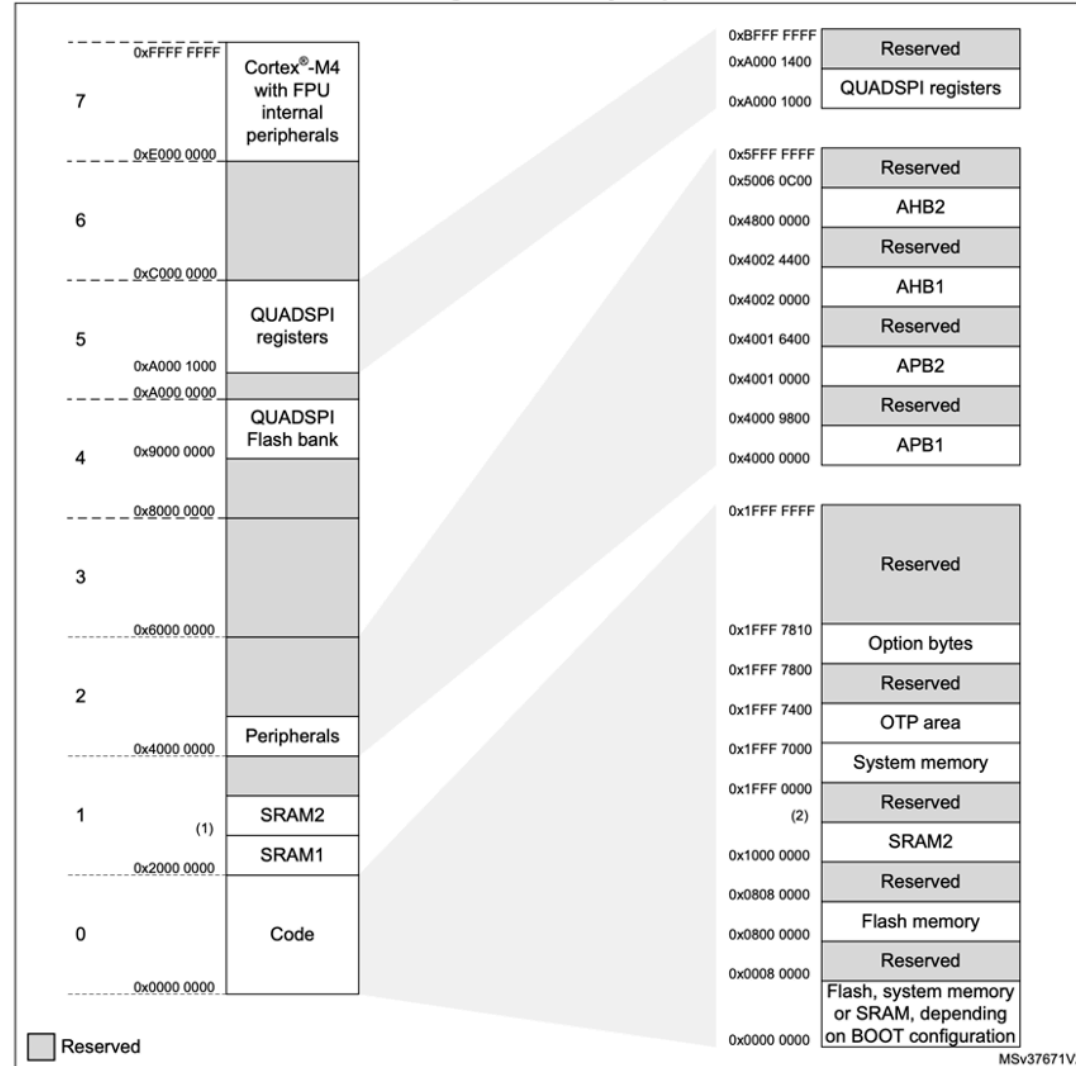


PM0214 Cortex-M4 Programmers Manual

Memory Map - STM32L432KC

RM0394 p. 67

Figure 2. Memory map



Part 3: From the Manual to Working C

Important Concepts in C

- C is libertarian by nature. You can stomp on any memory address you want!
- There is no memory management built in. You must manually allocate (and deallocate!) any memory you need.
- On an MCU this is the whole point. A peripheral register *is* a memory address, so writing to a pointer is how you configure hardware.

Primitive Data Types in `stdint.h`

Table eC.2 Primitive data types and sizes

Type	Size (bits)	Minimum	Maximum
char	8	$-2^{-7} = -128$	$2^7 - 1 = 127$
unsigned char	8	0	$2^8 - 1 = 255$
int	machine-dependent		
unsigned int	machine-dependent		
int16_t	16	$-2^{15} = -32,768$	$2^{15} - 1 = 32,767$
uint16_t	16	0	$2^{16} - 1 = 65,535$
int32_t	32	$-2^{31} = -2,147,483,648$	$2^{31} - 1 = 2,147,483,647$
uint32_t	32	0	$2^{32} - 1 = 4,294,967,295$
int64_t	64	-2^{63}	$2^{63} - 1$
uint64_t	64	0	$2^{64} - 1$
float	32	$\pm 2^{-126}$	$\pm 2^{127}$
double	64	$\pm 2^{-1023}$	$\pm 2^{1022}$

A control register is a fixed 32 bits wide. Say so: use `uint32_t`, never `int` or `unsigned long`, whose widths are not guaranteed.

Important Keywords in C

- `volatile` – prevents the compiler from using a cached value (forces load)
- `const` – “read-only”. Prevents you from assigning a value to the variable.
- `static`
 - Inside a function: retains its values between calls.
 - Applied to a function: visible only in this file
- `extern`
 - Applied to a function definition: has global scope (redundant)
 - Applied to a variable: defined elsewhere
- `void`
 - As return type of function: doesn't return a value
 - In a pointer declaration, the type of a generic pointer
 - In a parameter list: takes no parameters

`volatile` is the one that will bite you in Lab 4. Hardware changes a status register behind the compiler's back; without it, `while (!(TIM2->SR & 1));` gets optimized into an infinite loop.

C Idioms for Setting and Clearing Bits

```
1 #define GPIOA_BASE 0x48000000
2 #define GPIOA_MODER (*((volatile unsigned long *) (GPIOA_BASE + 0x00)))
3
4 // Set bit 3 of the GPIOA_MODER to 1.
5
6
7 // Clear bit 7 of the GPIOA_MODER (i.e., set to 0)
8
```



- ① `GPIOA_MODER |= (1 << 3);`
- ② `GPIOA_MODER &= ~(1 << 7);`

Pointers and Arrays in C: Arrays

```
1 int * p = (int*) 0x20000000;  
2  
3 int a = *p;  
4  
5 int b = *(p+3);  
6  
7 *(p+5) = b;
```

①

②

③

- ① Equivalent to `a = p[0]`
- ② Equivalent to `b = p[3]`, address `0x2000000C`
- ③ Equivalent to `p[5] = b`, address `0x20000014`

Structures

```
1 struct contact {  
2     char name[30];  
3     unsigned long long phone;  
4     float height;  
5 };  
6  
7 struct contact jbrake; // example variable definition
```

How many bytes does this structure occupy in memory?

$30 * \text{sizeof}(\text{char}) + \text{sizeof}(\text{unsigned long long}) + \text{sizeof}(\text{float})$

$= 30 \text{ B} + 8 \text{ B} + 4 \text{ B} = 42 \text{ Bytes}$

A Struct Is a Register Map

The registers of a peripheral are consecutive words in memory. A struct whose fields are `uint32_t` lays out exactly the same way, so one struct describes the whole peripheral.

```
1 typedef struct {  
2     volatile uint32_t MODER;    // offset 0x00  
3     volatile uint32_t OTYPER;   // offset 0x04  
4     volatile uint32_t OSPEEDR;  // offset 0x08  
5     volatile uint32_t PUPDR;    // offset 0x0C  
6     volatile uint32_t IDR;      // offset 0x10  
7     volatile uint32_t ODR;      // offset 0x14  
8 } GPIO_TypeDef;
```

Read the offsets straight out of RM0394 §8.4. Every field must be declared, in order, even ones you never use – they are what puts the later fields at the right addresses.

Arrow Operator

Point the struct at the peripheral's base address and the compiler does the offset arithmetic for you.

```
1 #define GPIOA ((GPIO_TypeDef *) 0x48000000UL)
2
3 GPIOA->ODR |= (1 << 5);
```

- ① Base address from the register boundary table, RM0394 p. 68
- ② Writes to address `0x48000014`, because `ODR` is the `sixth uint32_t` in the struct

This is exactly what CMSIS does for you. Lab 4 asks you to write it yourself so that you have done it once, from the manual, before you let a header do it.

Wrap Up

- Every vendor splits the same information the same way. Find the document that answers your question before you start searching inside it.
- C is libertarian – will allow you to do many things, not all of which are good for you.
- Understanding certain C data structures like pointers and structures will enable you to more easily and naturally write code to control your MCU.
- Next time: GPIO and clock configuration, the two peripherals you need before Lab 4 makes a sound.

Appendix: Reference Slides

Microarchitecture Flashback

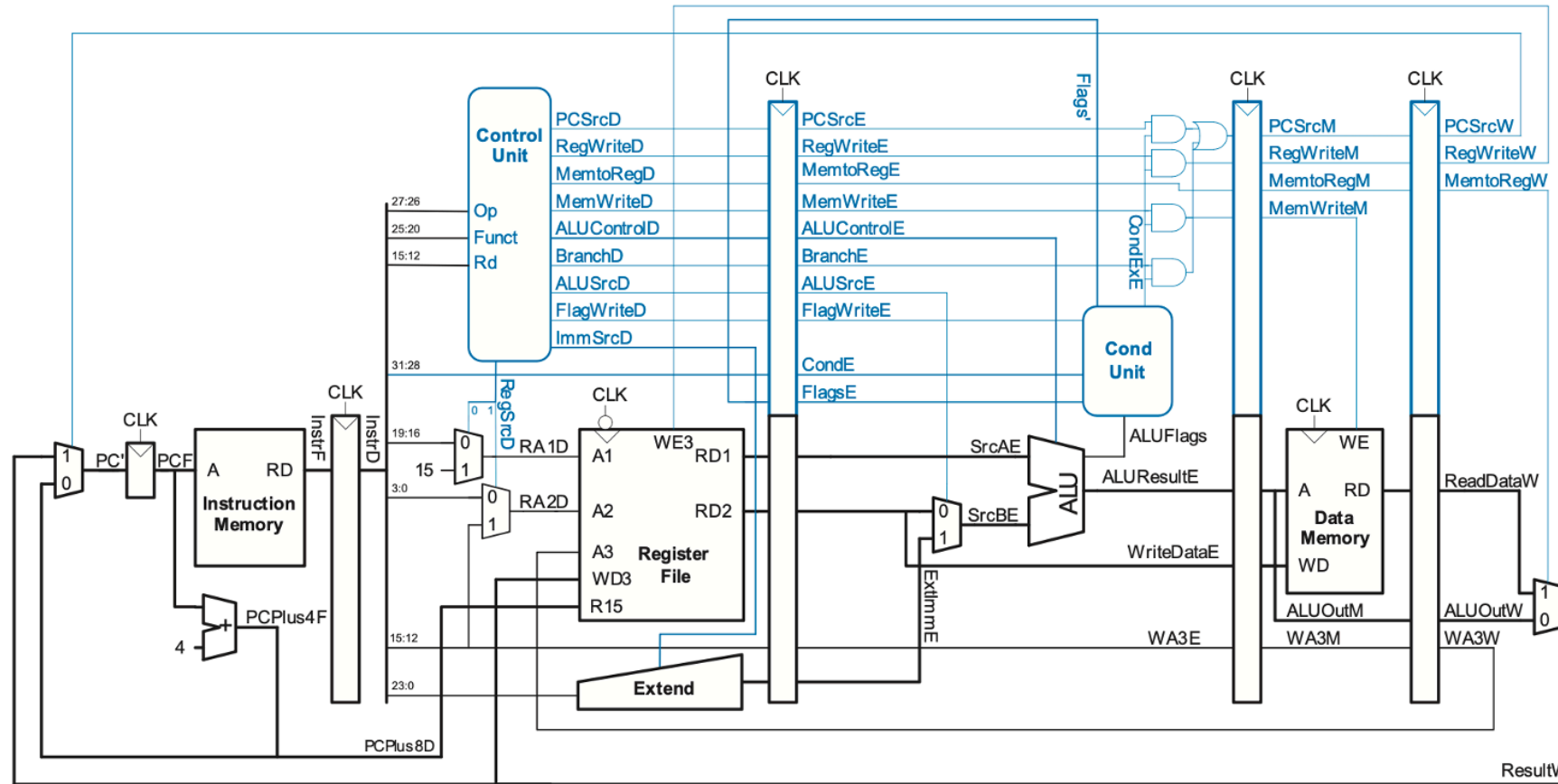


Figure 7.47 Pipelined processor with control

Harris and Harris, *Digital Design and Computer Architecture ARM Ed.*, p. 430

Register Set

- R15 is **Program Counter (PC)**
- R14 is **Link Register (LR)**. Holds return addresses
- R13 by convention used as the **stack pointer**.
- Four condition codes in current program status register (CPSR)
 - N – **negative**
 - Z – **zero**
 - C – **carry (unsigned overflow)**
 - V – **(signed) overflow**

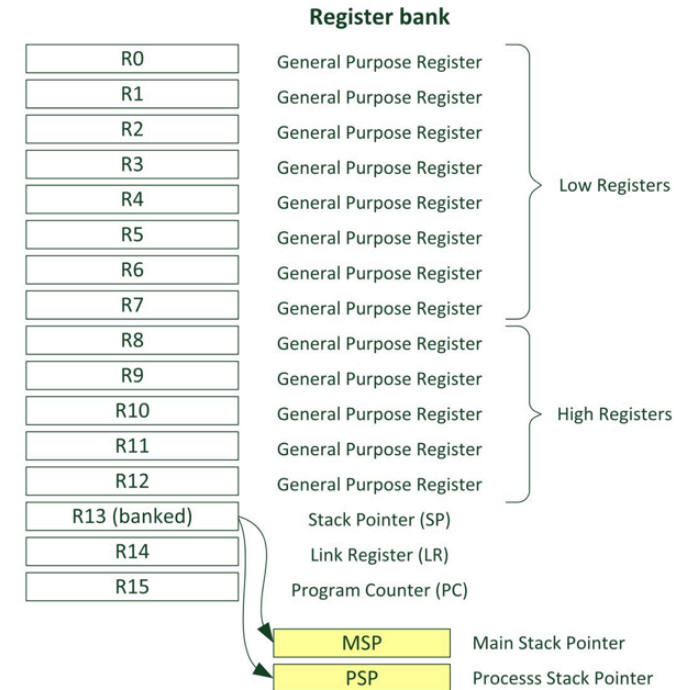


FIGURE 4.3

Registers in the register bank

Primitive Data Types in C

Table eC.2 Primitive data types and sizes

Type	Size (bits)	Minimum	Maximum
char	8	$-2^{-7} = -128$	$2^7 - 1 = 127$
unsigned char	8	0	$2^8 - 1 = 255$
short	16	$-2^{15} = -32,768$	$2^{15} - 1 = 32,767$
unsigned short	16	0	$2^{16} - 1 = 65,535$
long	32	$-2^{31} = -2,147,483,648$	$2^{31} - 1 = 2,147,483,647$
unsigned long	32	0	$2^{32} - 1 = 4,294,967,295$
long long	64	-2^{63}	$2^{63} - 1$
unsigned long long	64	0	$2^{64} - 1$
int	machine-dependent		
unsigned int	machine-dependent		
float	32	$\pm 2^{-126}$	$\pm 2^{127}$
double	64	$\pm 2^{-1023}$	$\pm 2^{1022}$

Operators and Operator Precedence

Table eC.3 Operators listed by decreasing precedence

Category	Operator	Description	Example
Unary	++	post-increment	a++; // a = a+1
	--	post-decrement	x--; // x = x-1
	&	memory address of a variable	x = &y; // x = the memory // address of y
	~	bitwise NOT	z = ~a;
	!	Boolean NOT	!x
	-	negation	y = -a;
	++	pre-increment	++a; // a = a+1
	--	pre-decrement	--x; // x = x-1
	(type)	casts a variable to (type)	x = (int)c; // cast c to an // int and assign it to x
	sizeof()	size of a variable or type in bytes	long int y; x = sizeof(y); // x = 4
Multiplicative	*	multiplication	y = x * 12;
	/	division	z = 9 / 3; // z = 3
	%	modulo	z = 5 % 2; // z = 1
Additive	+	addition	y = a + 2;
	-	subtraction	y = a - 2;
Bitwise Shift	<<	bitshift left	z = 5 << 2; // z = 0b00010100
	>>	bitshift right	x = 9 >> 3; // x = 0b00000001
Relational	==	equals	y == 2
	!=	not equals	x != 7
	<	less than	y < 12
	>	greater than	val > max
	<=	less than or equal	z <= 2
	>=	greater than or equal	y >= 10

Table eC.3 Operators listed by decreasing precedence—Cont'd

Category	Operator	Description	Example
Bitwise	&	bitwise AND	y = a & 15;
	^	bitwise XOR	y = 2 ^ 3;
		bitwise OR	y = a b;
Logical	&&	Boolean AND	x && y
		Boolean OR	x y
Ternary	? :	ternary operator	y = x ? a : b; // if x is TRUE, // y=a, else y=b
Assignment	=	assignment	x = 22;
	+=	addition and assignment	y += 3; // y = y + 3
	-=	subtraction and assignment	z -= 10; // z = z - 10
	*=	multiplication and assignment	x *= 4; // x = x * 4
	/=	division and assignment	y /= 10; // y = y / 10
	%=	modulo and assignment	x %= 4; // x = x % 4
	>>=	bitwise right-shift and assignment	x >>= 5; // x = x >> 5
	<<=	bitwise left-shift and assignment	x <<= 2; // x = x << 2
	&=	bitwise AND and assignment	y &= 15; // y = y & 15
	=	bitwise OR and assignment	x = y; // x = x y
	^=	bitwise XOR and assignment	x ^= y; // x = x ^ y

Operator Precedence Tip!

You should only have to remember multiplication/division before addition/subtraction.
For everything else, use parentheses!

Important Libraries

- `stdint.h` – standard fixed-width types (e.g., `uint32_t`)
- `stdio.h` – standard input and output. Contains functions like `printf` or `fprintf`.
- `stdlib.h` – standard library: random number generation (`rand` and `srand`), allocating or freeing memory (`malloc` and `free`).
- `math.h` – math library: standard math functions like `sin`, `cos`, `sqrt`, `log`, `exp`, `floor`, `ceil`.
- `string.h` – string library: functions to compare, copy, concatenate, and determine the length of a string.

Pointers and Arrays in C: Strings

```
1 char * str = (char *) 0x20001000;  
2  
3 str[13] = 'A';
```

①

① Address `0x2000100D` = `0x41`

Dereferencing

```
1
2 int * p = (int*) 0x20000000;
3
4 int a = *p;
5 int * aptr = &a;
6
7 *aptr = 3;
8
9 int * ptr = &p[0];
10
11 ptr = &p[5]
12
13 *ptr = 42;
```

①
②
③
④
⑤
⑥

- ① Equivalent to `a = p[0]`
- ② `aptr` stores address of `a`
- ③ same as `a=3`
- ④ `ptr= 0x20000000`
- ⑤ `ptr= 0x20000014`
- ⑥ `p[5]= 42`

Structure Declaration Syntax

```
1 struct optional_tag {  
2     type_1 identifier_1;  
3     type_2 identifier_2;  
4     ...  
5     type_N identifier_N;  
6 } optional_variable_definitions ;
```

Using structures as part of a new type

Can also wrap in a typedef to avoid needing to use the `struct` keyword.

```
1 typedef struct my_tag {int i;} my_type; // Declaration of new type
2
3 // Creating variable with struct keyword
4
5
6
7 // Creating variable using new type
8
```

- ① `struct my_tag variable_1;`
- ② `my_type variable_2;`